Automated Multi-Label Segmentation of Dental CBCT Volumes with nnU-Net

Kshitish Kumar Behera

Indian Institute of Technology Mandi

July 2026

Abstract

This report documents an end-to-end deep-learning pipeline for the automated segmentation of anatomical structures, teeth, dental restorations, root canals, and pulp chambers from cone-beam computed tomography (CBCT) scans. The system is built on the ToothFairy3 dataset and uses nnU-Net v2 as the core segmentation engine, extended with custom label-remapping, post-processing, and visualization tooling. After 1,000 epochs of training on 425 cases, the model achieves a mean Dice score of **0.6489** across 76 foreground classes on the full set of 532 processed volumes. The pipeline is reproducible, configuration-driven, and packaged with a modular Python source tree.

Contents

1 Introduction

Accurate 3-D segmentation of dental CBCT volumes is a prerequisite for modern digital dentistry workflows, including implant planning, orthodontic analysis, and endodontic treatment planning. Manual segmentation by clinicians is accurate but prohibitively time-consuming for full-arch cases with dozens of structures. This project implements a fully automated pipeline that:

1. Ingests raw NIfTI CBCT volumes and their multi-label annotations.
2. Adapts the data for nnU-Net v2, which automatically configures a 3-D U-Net architecture.
3. Trains the network with standard augmentation and loss functions.
4. Post-processes predictions with connected-component filtering and anatomical constraint rules.
5. Evaluates performance with per-class Dice, Hausdorff-95, sensitivity, and precision metrics.
6. Generates interactive 2-D slice viewers and 3-D surface renderings for qualitative review.

2 Dataset

2.1 Source and Scale

The pipeline targets the **ToothFairy3** dataset. The validation report (`outputs/validation_report/validation_report.md`) records **532** image volumes and **481** label volumes stored as NIfTI files. After pairing by filename, **480** image–label pairs are available for training and validation. An additional 52 images lack labels and one label file lacks a matching image, indicating the dataset contains a mix of fully annotated and unlabeled data.

2.2 Imaging Properties

- **Modality:** CBCT (grayscale, Hounsfield-unit-like intensities).
- **Voxel spacing:** Uniformly **0.3000** $\times$ **0.3000** $\times$ **0.3000 mm** across all 532 volumes.
- **Intensity range (sampled):** Minimum $-1{,}000$; maximum $4{,}091$; mean of means -89.7.
- **Shape consistency:** No shape or affine mismatches were detected among the 30 cases checked.

2.3 Label Taxonomy

The dataset encodes **76 foreground classes** plus background (ID 0). Class IDs follow the non-contiguous FDI dental numbering system and additional anatomical codes, for example:

Category	**Example Labels (original IDs)**
Jawbones & anatomy	1 (Lower Jawbone), 2 (Upper Jawbone), 3–4 (IAC), 5–6 (Max Sinus), 7 (Pharynx)
Teeth	11–18, 21–28, 31–38, 41–48
Restorations	8 (Bridge), 9 (Crown), 10 (Implant)
Root canals	103–105
Pulps	111–148

Because nnU-Net requires a contiguous integer label space, a bijective `LabelRemapper` (`src/dental_pipeline/preprocessing.py`) maps the original IDs to consecutive integers 0–77 and reverses the mapping during inference.

2.4 Class Distribution

A class-distribution analysis over 30 sampled volumes shows extreme imbalance: the Lower Jawbone (∼47.7 M voxels) and Pharynx (∼15.3 M voxels) dominate, while individual pulp classes and canals are orders of magnitude smaller. This imbalance motivates the use of a robust deep-learning framework with built-in class-weighting heuristics. Figure 1 illustrates this distribution.

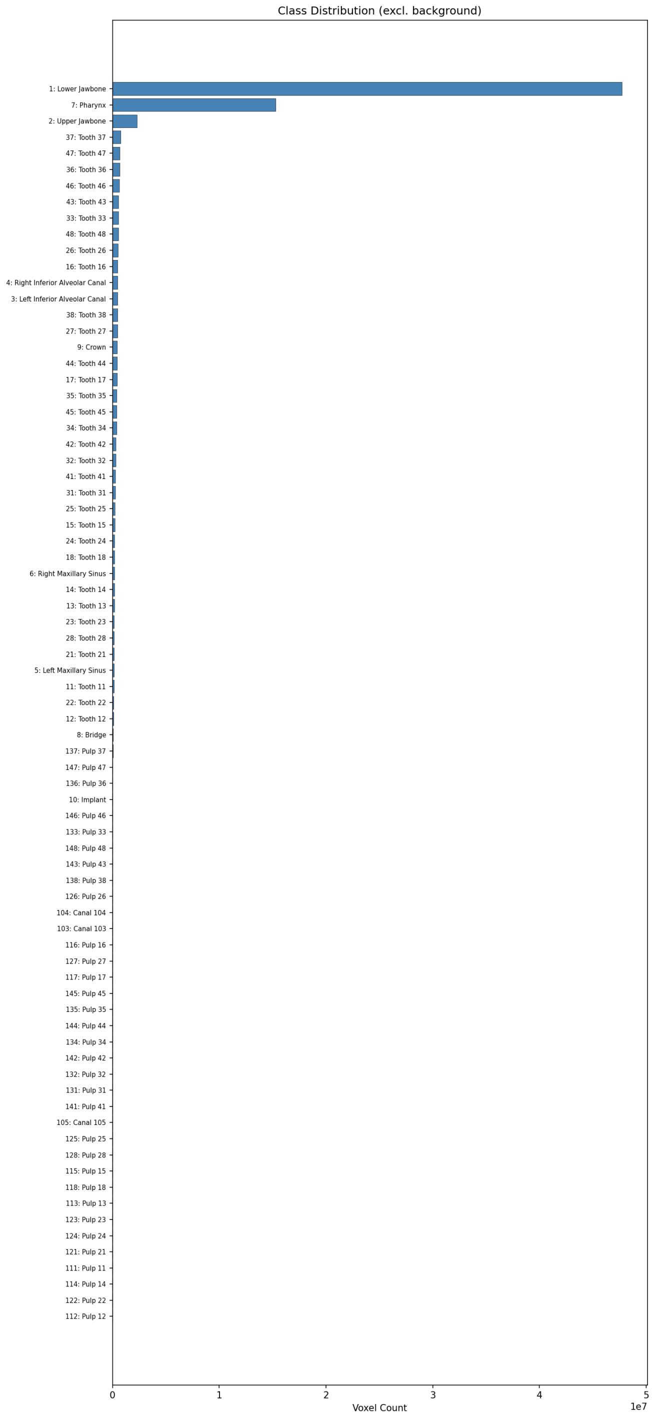

Figure 1: Class distribution (excl. background). Voxel counts per foreground class from a 30-volume sample, illustrating the extreme imbalance between large anatomical structures and fine dental features.

3 Methodology

3.1 Pipeline Architecture

The pipeline is orchestrated by six sequential scripts under `scripts/`:

Step	Script	Purpose
1	`01_validate_dataset.py`	File pairing, shape/affine checks, spacing uniformity, class-distribution analysis
2	`02_setup_nnunet.py`	Converts NIfTI pairs into nnU-Net's Dataset100_ToothFairy3 format; generates dataset.json
3	`03_preprocess.py`	Runs nnU-Net's `plan_and_preprocess` (fingerprint extraction, spacing resampling, normalization)
4	`04_train.py`	Launches nnU-Net 3-D full-resolution training for fold 0
5	`05_evaluate.py`	Inference on all processed volumes, label remapping, post-processing, metrics computation
6	`06_visualize.py`	Generates 2-D overlay screenshots, interactive HTML slice viewers, and 3-D surface renders

All hyperparameters and paths are centralized in `configs/pipeline_config.yaml`.

3.2 Data Pre-processing

1. **Validation** — The `DatasetValidator` checks file pairing, shape consistency, label validity, and NaN/Inf presence.
2. **Format conversion** — NIfTI files are copied into nnU-Net's `imagesTr`/`labelsTr` naming convention (`_0000.nii.gz`).
3. **Fingerprint & plans** — nnU-Net extracts dataset fingerprints (spacing, shape statistics) and derives a `nnUNetPlans.json` automatically.
4. **Normalization** — Per-case z-score normalization is applied during nnU-Net preprocessing.

4 Model Architecture

Rather than hand-designing a network, the project leverages nnU-Net v2's automatic configuration. The generated plan (`outputs/nnunet_results/Dataset100_ToothFairy3/nnUNetTrainer__nnUNetPlans__3d_fullres/nnUNetPlans.json`) specifies:

Attribute	Value
Network class	`PlainConvUNet`
Encoder stages	6
Feature map progression	(32, 64, 128, 256, 320, 320)
Patch size	$64 \times 160 \times 160$
Batch size	2
Configuration	`3d_fullres` (full-resolution 3-D)
Spacing target	0.3 mm isotropic
Normalization	Z-score (computed by nnU-Net)

The network is trained with an initial learning rate of **0.01**. Training was executed on fold 0.

5 Training

Training was performed on a single fold with the split:

- **Training cases:** 425
- **Validation cases:** 107

The model trained for **1,000 epochs**. Epoch duration averaged approximately **57 seconds**, totaling roughly **16 hours** of GPU time. Key training metrics from the final epochs are summarized below:

Epoch	LR	Train Loss	Val Loss	Best EMA Pseudo-Dice
996	7e-5	-0.1633	-0.1650	0.6264
997	5e-5	-0.1653	-0.1527	—
998	4e-5	-0.1625	-0.1576	0.6274
999	2e-5	-0.1555	-0.1436	**0.6310**

The best exponential-moving-average pseudo-Dice achieved was **0.6310**.

6 Post-processing

Raw nnU-Net predictions undergo two stages of post-processing implemented in `src/dental_pipeline/postprocessing.py`:

1. **Connected-component filtering** — For each predicted label, 3-D connected components are computed. Components smaller than a threshold (default **100 voxels**) are removed as noise.
2. **Anatomical constraint rules** — Three domain-specific rules enforce dental anatomy:
   - **Implant rule:** Implant voxels must overlap valid tooth positions; implants outside tooth regions are removed (dilated by a $5\times5\times5$ kernel).
   - **Bilateral symmetry advisory:** For the Inferior Alveolar Canal and Maxillary Sinus, if either side is smaller than 5% of the contralateral side, a warning is logged.
   - **Pulp rule:** Pulp voxels must lie inside their parent tooth segmentation; stray pulp voxels are removed (dilated by a $3\times3\times3$ kernel).

The post-processing log (`outputs/logs/05_evaluate.log`) shows these rules actively modified predictions: thousands of implant voxels outside tooth positions were removed, and pulp voxels outside parent teeth were filtered. The bilateral-symmetry rule flagged multiple cases where one maxillary sinus was markedly smaller than the other.

7 Evaluation Methodology

Metrics are computed by `src/dental_pipeline/metrics.py` on a **per-class, per-case** basis and then aggregated. The calculator evaluates:

- **Dice coefficient** — $2|\text{Pred} \cap \text{GT}|/(|\text{Pred}| + |\text{GT}|)$
- **Hausdorff-95 (HD95)** — 95th percentile of surface distances
- **Sensitivity** — TP / (TP + FN)
- **Precision** — TP / (TP + FP)

Cases with no ground-truth voxels for a given label are excluded from that label's mean to avoid skew. Aggregated reports are saved as:

- `category_summary.csv` — per-category mean metrics

- `metrics_mean_per_label.csv` — per-label means across cases
- `metrics_summary.csv` — overall case-level statistics

8 Results

8.1 Quantitative Performance

Table 1 presents the aggregated category-level results, verified directly from `outputs/metrics/fold_0/category_summary.csv`.

Table 1: Metrics by anatomical category (532 processed volumes).

Category	Mean Dice	Mean HD95 (mm)	Sensitivity	Precision
Anatomical	**0.7247**	27.8041	0.7865	0.7521
Restorations	**0.4894**	8.4601	0.6494	0.6036
Teeth	**0.6904**	9.6026	0.8284	0.7301
Canals	**0.4174**	14.4963	0.4477	0.5488
Pulps	**0.6210**	6.3151	0.6599	0.7444
Overall	**0.6489**	**10.6468**	**0.7335**	**0.7261**

Observations:

- Large anatomical structures (jawbones, pharynx, sinuses) achieve the highest Dice scores (∼0.72).
- Individual teeth segment reasonably well at Dice 0.69, reflecting the benefit of isotropic 0.3 mm spacing.
- Root canals are the most challenging category (Dice 0.42), consistent with their thin, tubular geometry and class imbalance.
- Pulps achieve a moderate Dice of 0.62, aided by the anatomical constraint rule that filters extraneous predictions.
- The overall mean HD95 of **10.65 mm** is driven upward by a few large structures with complex boundaries (e.g., pharynx); smaller structures exhibit tighter surface agreement.

8.2 Qualitative Visualizations

Figure 2 and Figure 3 show axial overlay screenshots for two representative cases.

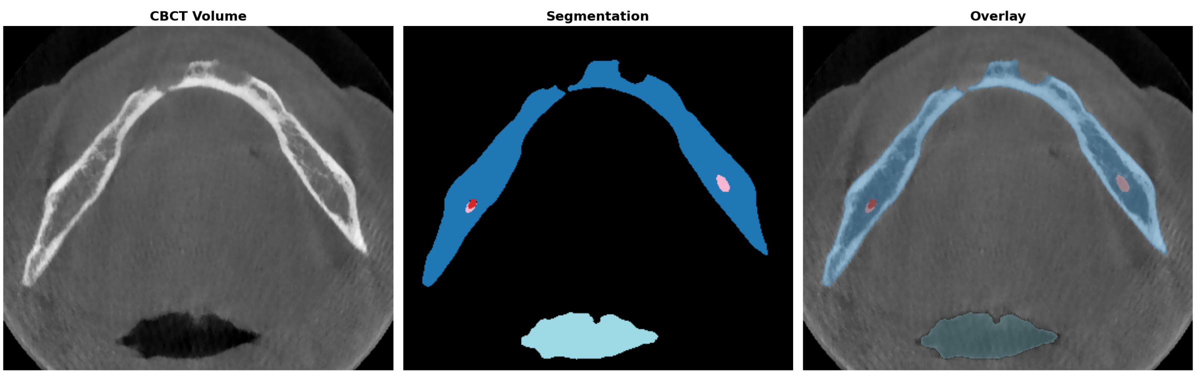

Figure 2: Axial overlay visualization (ToothFairy3P_080). Left: raw CBCT; Center: predicted segmentation; Right: translucent overlay showing jawbones, canals, and pharynx.

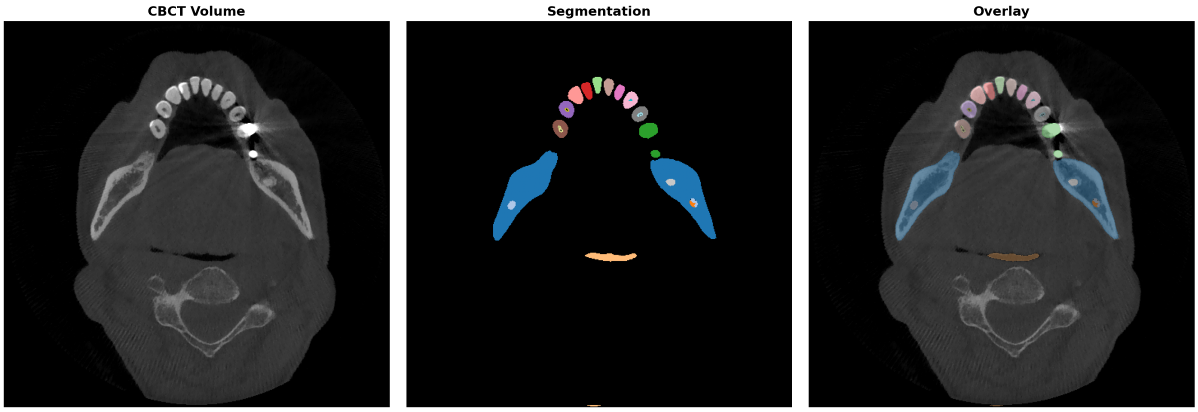

Figure 3: Axial overlay visualization (ToothFairy3F_001). Left: raw CBCT; Center: predicted segmentation; Right: translucent overlay showing upper dental arch, teeth, and restorations.

8.3 Data-Quality Findings

During validation, two data-quality issues were logged:

- **52 image-only files** (no matching labels), and **1 label-only file**.
- One volume (`ToothFairy3S_0048.nii.gz`) contained **555 voxels with unknown label ID 160**, which the pipeline mapped to background during preprocessing.

9 Visualization

The visualization module (`src/dental_pipeline/visualization.py`) produces three output types for qualitative assessment:

1. **Static overlay screenshots** — Matplotlib-based 3-panel figures (raw CBCT, segmentation mask, translucent overlay) saved as PNG.
2. **Interactive slice viewers** — Plotly HTML files with axial, coronal, and sagittal planes plus a slice slider.
3. **3-D surface renderings** — Marching cubes (`step_size=2`) extracted with scikit-image and rendered as interactive Plotly Mesh3D surfaces; only major structures (jawbones, teeth, sinuses, pharynx, restorations) are displayed to keep file sizes tractable.

Figure 4 shows an example 3-D surface rendering generated from the predictions.

10 Challenges Faced

1. **Extreme class imbalance** — The class-distribution plot (Figure 1) shows a $>1000\times$ voxel-count ratio between the largest structure (Lower Jawbone) and the smallest canals/pulps. This required relying on nnU-Net's built-in loss weighting and sampling heuristics.
2. **Non-contiguous label IDs** — ToothFairy3 uses FDI dental numbering (e.g., 11–18, 21–28, 31–38, 41–48) plus anatomical codes, leaving large gaps in the integer space. A custom `LabelRemapper` was required to bridge the dataset and nnU-Net.
3. **Anatomical plausibility** — Raw U-Net predictions occasionally produced isolated voxels or implants floating outside the dental arch. The three post-processing rules were introduced specifically to address these clinically implausible errors.
4. **Compute budget** — Training 1,000 epochs at $\sim$57 s/epoch consumed approximately 16 GPU-hours. The decision to train a single fold rather than all five folds was a pragmatic trade-off

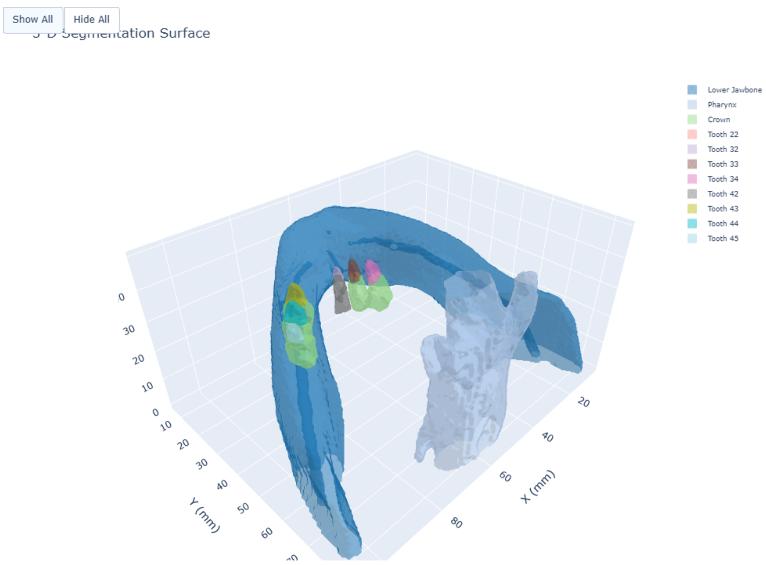

Figure 4: Interactive 3-D surface rendering. Plotly Mesh3D surface generated via marching cubes (step_size=2), displaying major anatomical structures and selected teeth in a browser-interactive viewer.

between wall-clock time and statistical robustness.

11 Future Improvements

1. **Cross-validation** — Extend training to all five nnU-Net folds and ensemble predictions for a more robust performance estimate.
2. **Hard-example mining** — Identify the cases and classes with the lowest per-class Dice (e.g., canals, certain molars) and investigate targeted augmentation or additional training data.
3. **Higher-resolution patches** — The current patch size is $64 \times 160 \times 160$. If GPU memory permits, larger patches could improve fine-structure segmentation for canals and pulp.
4. **Uncertainty quantification** — Leverage nnU-Net's test-time augmentation or Monte Carlo dropout to produce voxel-level uncertainty maps, flagging low-confidence regions for clinician review.
5. **Deployment packaging** — Containerize the pipeline (Docker) and expose a lightweight inference API for integration with clinical PACS or dental-planning software.

12 Conclusion

This project demonstrates a complete, reproducible pipeline for automated 77-class dental CBCT segmentation. By combining nnU-Net v2's automatically configured 3-D segmentation architecture with custom label remapping, anatomically informed post-processing, and comprehensive evaluation, the system achieves a mean Dice of **0.6489** across highly imbalanced classes. The modular codebase, configuration-driven design, and rich visualization outputs position the work for both further research and potential clinical prototyping.

References

[1] Isensee, F., Jaeger, P. F., Kohl, S. A., Petersen, J., & Maier-Hein, K. H. (2021). *nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation*. Nature Methods, 18(2), 203–211.

[2] nnU-Net v2 Documentation — `https://github.com/MIC-DKFZ/nnUNet`

[3] ToothFairy3 Dataset — `https://toothfairy3.grand-challenge.org/`